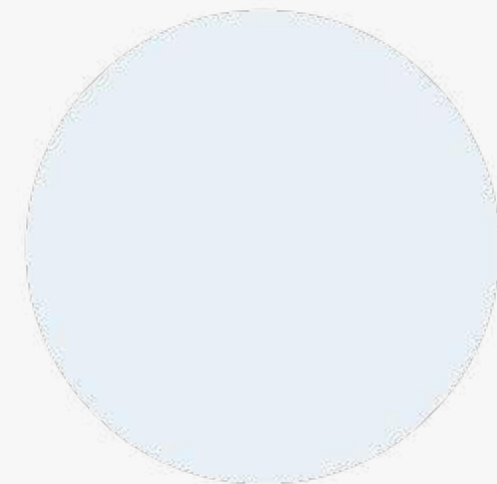


ASTON.



Фреймворки для автоматизации тестирования



astondevs.ru

Понятие и назначение



Фреймворк автоматизации тестирования - это интегрированная система, которая устанавливает правила автоматизации конкретного продукта. Эта система объединяет библиотеки функций, источники тестовых данных, сведения об объектах и различные повторно используемые модули. Эти компоненты действуют как небольшие строительные блоки, которые необходимо собрать для представления бизнес-процесса. Платформа обеспечивает основу для автоматизации тестирования и упрощает процесс автоматизации.

Фреймворки автоматизации тестирования представляют собой не что иное, как успешно развившиеся и ставшие популярными решения, объединяющие в себе лучшие стороны тестирования под управлением данными, ключевыми словами и возможности реализации частных решений на высоком уровне абстракции.

Понятие и назначение



Фреймворков автоматизации тестирования очень много, они очень разные, но их объединяет несколько общих черт:

- высокая абстракция кода (нет необходимости описывать каждое элементарное действие) с сохранением возможности выполнения низкоуровневых действий;
- универсальность и переносимость используемых подходов;
- достаточно высокое качество реализации (для популярных фреймворков).

Как правило, каждый фреймворк специализируется на своём виде тестирования, уровне тестирования, наборе технологий. Существуют фреймворки для модульного тестирования (например, семейство xUnit), тестирования веб-приложений (например, семейство Selenium), тестирования мобильных приложений, тестирования производительности и т.д.

Существуют бесплатные и платные фреймворки, оформленные в виде библиотек на некотором языке программирования или в виде привычных приложений с графическим интерфейсом, узко- и широко специализированные и т.д.

Понятие и назначение



Преимущества:

- Широкое распространение;
- Универсальность в рамках своего набора технологий;
- Хорошая документация и большое сообщество специалистов, с которыми можно проконсультироваться;
- Высокий уровень абстракции;
- Наличие большого набора готовых решений и описаний соответствующих лучших практик применения того или иного фреймворка для решения тех или иных задач.

Понятие и назначение



Недостатки:

- Требуется время на изучение фреймворка;
- В случае написания собственного фреймворка де-факто получается новый проект по разработке ПО;
- Высокая сложность перехода на другой фреймворк;
- В случае прекращения поддержки фреймворка тест-кейсы рано или поздно придётся переписывать с использованием нового фреймворка;
- Высокий риск выбора неподходящего фреймворка.

JUnit 5

JUnit — самый популярный фреймворк для модульного тестирования Java-приложений, а последняя версия этого инструмента, т. е. JUnit 5, обладает ещё более полезным и мощным инструментарием.



Обзор



JUnit — одна из самых популярных платформ модульного тестирования для Java. Поэтому в сообществе разработчиков появляются большие проблемы, когда выходят новые основные версии.

Первая версия JUnit 5 вышла в далёком 2016 году. Но из-за недостатка документации и настроенной интеграции с основными решениями, используемыми в разработке на Java, довольно популярной остаётся версия JUnit 4. На нашем курсе мы рассмотрим последнюю, пятую, версию, так как новые проекты уже пишутся в основном на ней.

Созданный Кентом Беком и Эриком Гаммой, JUnit принадлежит семье фреймворков xUnit для разных языков программирования, берущей начало в SUnit Кента Бека для Smalltalk. JUnit породил экосистему расширений — jMock, EasyMock, DbUnit, HttpUnit и т. д.

JUnit переведён на другие языки, включая PHP (PHPUnit), C# (NUnit), Python (PyUnit), Fortran (fUnit), Delphi (DUnit), Free Pascal (FPCUnit), Perl (Test::Unit), C++ (CppUnit), Flex (FlexUnit), JavaScript (JSUnit), COS (COSUnit).

Зависимости и настройка



Установка JUnit 5 довольно проста. Требуется просто добавить следующую зависимость в наш pom.xml:

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-engine</artifactId>
  <version>5.6.2</version>
  <scope>test</scope>
</dependency>
```

Многие проблемы с интеграцией JUnit 5 в действующих IDE решаются тяжело, поэтому применяется готовое решение для отчётов:

```
<build>
  <plugins>
    <plugin>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.0.0-M5</version>
    </plugin>
    <plugin>
      <artifactId>maven-failsafe-plugin</artifactId>
      <version>2.12</version>
    </plugin>
    <plugin>
      <artifactId>maven-surefire-report-plugin</artifactId>
      <version>3.0.0-M5</version>
    </plugin>
    <!-- Лечение проблемы для mvn site: A required class was missing while
    executing org.apache.maven.plugins:maven-site-plugin:3.3:site:
    org/apache/maven/doxia/siterenderer/DocumentContent-->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-site-plugin</artifactId>
      <version>3.7.1</version>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-project-info-reports-plugin</artifactId>
      <version>3.0.0</version>
    </plugin>
  </plugins>
</build>
```


Зависимости и настройка

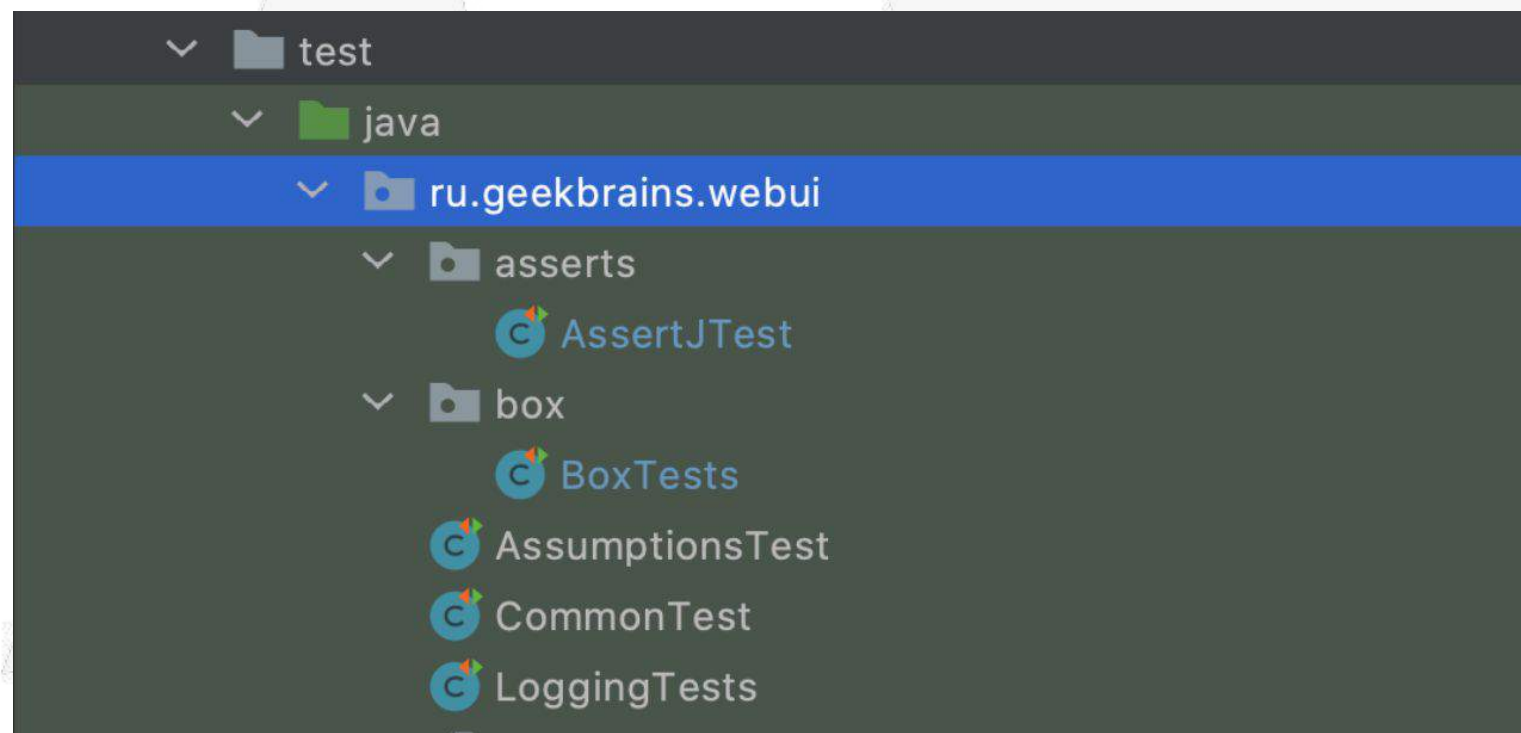


Важно отметить, что эта версия требует Java 8 для работы.
Чтобы получить SureFire-отчёты, требуется сначала
запустить тесты: `mvn clean install` или `mvn clean test` Затем —
запустить генерацию документации и отчёта: `mvn site
surefire-report:report`

Написание тестов



Тесты, где используется Maven для сборки проекта, должны лежать в пакете `src/test/java`. Внутри этого пакета тесты делятся на разные классы и даже пакеты для более удобного запуска:



Тесты по умолчанию должны находиться в классе, название которого начинается на `Test` или оканчивается на `Test`, `TestCase` или `Tests`. Тестом называется метод `public static void`, помеченный аннотацией `@org.junit.jupiter.api.Test` — здесь и далее все названия даются для JUnit 5.

Написание тестов



Тело теста традиционно делится на две части:

- функциональная часть (шаги теста) обычно находится прямо в проверке;
- проверки (asserts), где происходит сравнение актуального результата с ожидаемым.

Аннотации JUnit



Кроме аннотации `@Test`, которой обозначаются все тесты, есть ещё и другие. Для начала перечислим те, что используются для тестовых методов:

- `@DisplayName` (название метода) используется для обозначения методов в отчёте на понятном человеку языке.
- `@Disabled` (причина пропуска теста) — для пропуска, то есть исключения из прогона теста.
- `@Tag` (имя тега) тегирует тесты для фильтрации при запуске.
- `@RepeatedTest` (число) повторяет тест указанное количество раз.
- `@ParameterizedTest` используется вместо аннотации для передачи параметров в тест (`DataProvider` в `TestNG`).
- `@ValueSource(strings = { "s1", "s2" })` используется вместе с предыдущей аннотацией для объявления конкретных параметров для теста. Сейчас поддерживаются только данные примитивных типов: `int`, `long`, `double`, `String`.

Параметризованные тесты



Поговорим подробнее о параметризованных тестах. Кроме указанных выше аннотаций, в аргументах теста требуется указать название используемого параметра как в обычном методе:

```
@DisplayName("Слово является палиндромом")
@ParameterizedTest
@ValueSource(strings = { "racecar", "radar", "able was I ere I
saw elba" })
public void isPalindromeTest(String word){
    Assertions.assertTrue(functions.isPalindrome(word));
}
```

Хотя в @ValueSource можно использовать лишь примитивы, есть другие варианты указания источника данных:

Пример с разбором CSV

```
@ParameterizedTest
@CsvSource({ "foo, 1", "bar, 2", "'baz, qux', 3" })
// или даже так: @CsvFileSource(resources = "/two-column.csv") void
testWithCsvSource(String first, int second) {
    assertNotNull(first);
    assertEquals(0, second);
}
```


Параметризованные тесты



Пример с Enum

```
@ParameterizedTest
@EnumSource(value = TimeUnit.class, names = { "DAYS", "HOURS" })
void testWithEnumSourceInclude(TimeUnit timeUnit) {
    assertTrue(EnumSet.of(TimeUnit.DAYS,
        TimeUnit.HOURS).contains(timeUnit));
}
```

Пример с источником данных

```
@ParameterizedTest
@ArgumentsSource(MyArgumentsProvider.class)
void testWithArgumentsSource(String argument) {
    assertNotNull(argument);
}

static class MyArgumentsProvider implements ArgumentsProvider {
    @Override
    public Stream<? extends Arguments>
    provideArguments(ExtensionContext
    context) {
        return Stream.of("foo", "bar").map(Arguments::of);
    }
}
```

Подготовка данных



Для подготовки и удаления тестовых данных используются методы `@BeforeAll` (`@AfterAll`) — запускаются перед (после) всеми тестами один раз. Есть также методы `@BeforeEach` (`@AfterEach`), которые запускаются перед (после) каждым тестом.

Методы `@Before...` обычно называют методами `setUp()`, а методы `@After...` — `tearDown()`.

Для `@BeforeAll` и `@AfterAll` важно, чтобы они были объявлены как `static`, так как его инициализация происходит до остальных методов.

Полный список аннотаций JUnit есть в официальной документации.

Утверждения (Assertions)



Утверждения находятся в `org.junit.jupiter.api.Assertions` и были значительно улучшены по сравнению с предыдущими версиями.

Самые простые assertions выглядят следующим образом: `Assertions.assertTrue(condition)` проверяет, что булево значение внутри скобок равно `true`.

`Assertions.assertFalse(condition)` проверяет, что булево значение внутри скобок равно `false`.

`Assertions.assertEquals(number1, number2)` проверяет, что два значения равны.

Примечательно, что методы реализуются для всех числовых типов.

Утверждения (Assertions)



В JUnit 5 может использоваться лямбда-выражение в утверждениях:

```
@Test
void lambdaExpressions() {
    assertTrue(Stream.of(1, 2, 3)
        .stream()
        .mapToInt(i -> i)
        .sum() > 5, () -> "Sum should be greater than 5");
}
```

Одно из преимуществ использования лямбда-выражения состоит в том, что оно лениво инициализируется. Это экономит время и ресурсы, если построение сообщения считается дорогостоящим.

Теперь есть возможность группировать утверждения путём использования `assertAll()`, который сообщит о любых неудачных утверждениях в группе через `MultipleFailuresError`:

```
@Test
void groupAssertions() {
    int[] numbers = {0, 1, 2, 3, 4};
    assertAll("numbers",
        () -> assertEquals(numbers[0], 1),
        () -> assertEquals(numbers[3], 3),
        () -> assertEquals(numbers[4], 1)
    );
}
```

Это означает, что безопаснее делать более сложные утверждения, поскольку мы сможем определить точное местоположение любого отказа.

Предположения



Допущения (предположения, assumptions) используются для запуска тестов только при соблюдении конкретных условий. Это обычно используется для внешних условий, которые требуются для правильной работы теста. Однако внешние условия не имеют прямого отношения к тому, что тестируется. Мы можем объявить предположение через `assumeTrue()`, `assumeFalse()` и `assumingThat()`

```
@Test
void trueAssumption() {
    assumeTrue(5 > 1);
    assertEquals(5 + 2, 7);
}

@Test
void falseAssumption() {
    assumeFalse(5 < 1);
    assertEquals(5 + 2, 7);
}

@Test
void assumptionThat() {
    String someString = "Just a string";
    assumingThat(
        someString.equals("Just a string"),
        () -> assertEquals(2 + 2, 4)
    );
}
```

Если предположение не выполняется, то генерируется `TestAbortedException`, и тест просто пропускается. Предположения также понимают лямбда-выражение.

Исключения



JUnit 5 улучшает поддержку исключений. Добавился метод `assertThrows()`, который проверяет, что выбрасывает выражение:

```
@Test
void shouldThrowException() {
    Throwable exception =
    assertThrows(UnsupportedOperationException.class, ()
-> {
        throw new UnsupportedOperationException("Not supported");
    });
    assertEquals(exception.getMessage(), "Not supported");
}
```

Теперь можно легко получить любую информацию, которая понадобится в исключении.

Testng



Как и в JUnit тесты описываются с помощью аннотаций. Для начала рассмотрим иерархию тестов. Все тесты принадлежат к какой-либо последовательности тестов (сьюту), включают в себя некоторое количество классов, каждый из которых может состоять из нескольких тестовых методов. При этом классы и тестовые методы могут принадлежать к определенной группе. Наглядно это выглядит так:

```
+ - suite/  
  +- test0/  
    +- class0/  
      +- method0(integration group)/  
      +- method1(functional group)/  
      +- method2/  
    +- class1  
      +- method3(optional group)/  
  +- test1/  
    +- class3(optional group, integration group)/  
      +- method4/
```

Testng



У каждого участника этой иерархии могут иметься before и after конфигураторы. Запускается это все в таком порядке:

```
+-- before suite/  
  +- before group/  
    +- before test/  
      +- before class/  
        +- before method/  
          +- test/  
            +- after method/  
              ...  
        +- after class/  
          ...  
      +- after test/  
        ...  
    +- after group/  
      ...  
+- after suite/
```

Testng



Рассмотрим определенный тест:

```
public class LocaleUtilsOldStyleTest {
    private final Map<String, Locale> parseLocaleData = new
    HashMap<String, Locale>();
    @BeforeClass
    private void setUp() {
        parseLocaleData.put(null, null);
        parseLocaleData.put("", LocaleUtils.ROOT_LOCALE);
        parseLocaleData.put("en", Locale.ENGLISH);
        parseLocaleData.put("en_US", Locale.US);
        parseLocaleData.put("en_GB", Locale.UK);
        parseLocaleData.put("ru", new Locale("ru"));
        parseLocaleData.put("ru_RU_xxx", new Locale("ru", "RU",
"xxx"));
    }
    @AfterTest
    void tearDown() {
        parseLocaleData.clear();
    }
    @Test
    public void testParseLocale() {
        for (Map.Entry<String, Locale> entry :
parseLocaleData.entrySet()) {
            final Locale actual =
LocaleUtils.parseLocale(entry.getKey());
            final Locale expected = entry.getValue();
            assertEquals(actual, expected);
        }
    }
}
```

Что можно отметить?



- Аннотации `@BeforeSuite`, `@AfterSuite` обозначают методы, которые выполняются единожды до/после выполнения всех тестов. Здесь удобно располагать какие-либо тяжелые настройки общие для всех тестов, например, здесь можно создать пул соединений с базой данных.
- Аннотации `@BeforeTest`, `@AfterTest` обозначают методы, которые выполняются единожды до/после выполнения теста(тот, который включает в себя тестовые классы, не путать с тестовыми методами). Здесь можно хранить настройки какой-либо группы взаимосвязанных сервисов, либо одного сервиса, если он тестируется несколькими тест-классами.
- Аннотации `@BeforeClass`, `@AfterClass` обозначают методы, которые выполняются единожды до/после выполнения всех тестов в классе, идентичны предыдущим, но применимы к тест-классам. Наиболее применим для тестирования какого-то определенного сервиса, который не меняет свое состояние в результате теста.

Что можно отметить?



- Аннотации `@BeforeMethod`, `@AfterMethod` обозначают методы, которые выполняются каждый раз до/после выполнения тестового метода. Здесь удобно хранить настройки для определенного бина или сервиса, если он не меняет свое состояние в результате теста.
- Аннотации `@BeforeGroups`, `@AfterGroups` обозначает методы, которые выполняются до/после первого/последнего теста принадлежащего к заданным группам.
- Аннотация `@Test` обозначает сами тесты. Здесь размещаются проверки. Также применима к классам

У всех этих аннотаций есть следующие параметры:



- `enabled` — можно временно отключить, установив значение в `false`
- `groups` — обозначает, для каких групп будет исполнен
- `inheritGroups` — если `true` (а по умолчанию именно так), метод будет наследовать группы от тест-класса
- `timeOut` — время, после которого метод «свалится» и потянет за собой все зависимые от него тесты
- `description` — название, используемое в отчете
- `dependsOnMethods` — методы, от которых зависит, сначала будут выполнены они, а затем данный метод
- `dependsOnGroups` — группы, от которых зависит
- `alwaysRun` — если установить в `true`, будет вызываться всегда независимо от того, к каким группам принадлежит, не применим к `@BeforeGroups`, `@AfterGroups`

Как видно из примера тест практически ничем не отличается от такого же теста на JUnit.

Параметризированные тесты



Напишем этот же тест другим способом:

```
public class LocaleUtilsTest extends Assert {  
    @DataProvider  
    public Object[][] parseLocaleData() {  
        return new Object[][] {  
            {null, null},  
            {"", LocaleUtils.ROOT_LOCALE},  
            {"en", Locale.ENGLISH},  
            {"en_US", Locale.US},  
            {"en_GB", Locale.UK},  
            {"ru", new Locale("ru")},  
            {"ru_RU_some_variant", new Locale("ru", "RU",  
"some_variant")},  
        };  
    }  
  
    @Test(dataProvider = "parseLocaleData")  
    public void testParseLocale(String locale, Locale expected) {  
        final Locale actual = LocaleUtils.parseLocale(locale);  
        assertEquals(actual, expected);  
    }  
}
```

Параметризированные тесты



Проще? Конечно, данные хранятся отдельно от самого теста.
Удобно? Конечно, можно добавлять тесты, добавляя всего
лишь строку в метод `parseLocaleData`.

Итак, как это работает?



- Объявляем тестовый метод со всеми нужными ему параметрами, например входные и ожидаемые данные. В нашем случае это строка, которую нужно распарсить в локаль и ожидаемая в результате локаль.
- Объявляем дата провайдер, хранилище данных для теста. Обычно это метод, возвращающий `Object[][]` либо `Iterator<Object[]>`, содержащий список параметров для определенного теста, например `{«en_US», Locale.US}`. Этот метод должен быть зааннотирован с помощью `@DataProvider`, в самом тесте он объявляется с помощью параметра `dataProvider` в аннотации `@Test`. Также можно указать имя(параметр `name`), если не указывать в качестве имени будет использоваться название метода.

Параметризированные тесты



Еще один пример, теперь разнесем данные и логику теста в разные классы:

```
public class LocaleUtilsTestData {
    @DataProvider(name = "getCandidateLocalesData")
    public static Object[][] getCandidateLocalesData() {
        return new Object[][]{
            {null, Arrays.asList(LocaleUtils.ROOT_LOCALE)},
            {LocaleUtils.ROOT_LOCALE, Arrays.asList(LocaleUtils.ROOT_LOCALE)},
            {Locale.ENGLISH, Arrays.asList(Locale.ENGLISH, LocaleUtils.ROOT_LOCALE)},
            {Locale.US, Arrays.asList(Locale.US, Locale.ENGLISH, LocaleUtils.ROOT_LOCALE)},
            {new Locale("en", "US", "xxx"), Arrays.asList(
                new Locale("en", "US", "xxx"), Locale.US, Locale.ENGLISH, LocaleUtils.ROOT_LOCALE)
            },
        };
    }
}

public class LocaleUtilsTest extends Assert {
    // other tests

    @Test(dataProvider = "getCandidateLocalesData", dataProviderClass = LocaleUtilsTestData.class)
    public void testGetCandidateLocales(Locale locale, List<Locale> expected) {
        final List<Locale> actual = LocaleUtils.getCandidateLocales(locale);
        assertEquals(actual, expected);
    }
}
```


Параметризированные тесты



В этом случае задаются параметры `dataProviderClass` и `dataProvider`. Метод, возвращающий тестовые данные должен быть `static`. Кроме описанного выше есть еще один способ параметризовать тесты.

Нужный метод аннотируется с помощью `@Parameters`, где указываются имена всех необходимых параметров. Некоторые из параметров можно зааннотировать с помощью `@Optional` с указанием значения по умолчанию (если не указать, то будут использоваться значения по умолчанию для примитивов, либо `null` для всех остальных типов). Значения параметров хранятся в конфигурации `TestNG` (которая будет рассмотрена позже). Пример:

```
public class ParameterizedTest extends Assert {
    private DataSource dataSource;
    @Parameters({"driver", "url", "username", "password"})
    @BeforeClass
    public void setUpDataSource (String driver, String url, @Optional("sa")
String username, @Optional String password) {
        // create datasource
        dataSource = ...
    }
    @Test
    public void testOptionalData () throws SQLException {
        dataSource.getConnection();
        // do some stuff
    }
}
```

Параметризированные тесты



В данном случае метод `setUpDataSource` будет принимать в качестве параметров настройки соединения с БД, причем параметры `username` и `password` опциональны, с заданными значениями по умолчанию. Очень удобно использовать с данными, общими для всех тестов(ну или почти всех), например, как в примере настройки соединения с БД. Ну и в завершение следует сказать пару слов о фабриках, которые позволяют создавать тесты динамически. Также, как и сами тесты, могут быть параметризованы с помощью `@DataProvider` либо `@Parameters`:

```
public class FactoryTest {
    @DataProvider
    public Object[][] tablesData() {
        return new Object[][] {
            {"FIRST_TABLE"},
            {"SECOND_TABLE"},
            {"THIRD_TABLE"},
        };
    }

    @Factory(dataProvider = "tablesData")
    public Object[] createTest(String table) {
        return new Object[] { new GenericTableTest(table) };
    }
}

public class GenericTableTest extends Assert {
    private final String table;
    public GenericTableTest(final String table) {
        this.table = table;
    }

    @Test
    public void testTable() {
        System.out.println(table);
        // do some testing staff here
    }
}
```

Многопоточность



Нужно проверить, как поведет себя приложение во многопоточном окружении? Можно сделать так, чтобы тесты исполнялись одновременно из нескольких потоков:

```
public class ConcurrencyTest extends Assert {
    private Map<String, String> data;
    @BeforeClass
    void setUp() throws Exception {
        data = new HashMap<String, String>();
    }
    @AfterClass
    void tearDown() throws Exception {
        data = null;
    }

    @Test(threadPoolSize = 30, invocationCount = 100, invocationTimeOut = 10000)
    public void testMapOperations() throws Exception {
        data.put("1", "111");
        data.put("2", "111");
        data.put("3", "111");
        data.put("4", "111");
        data.put("5", "111");
        data.put("6", "111");
        data.put("7", "111");
        for (Map.Entry<String, String> entry : data.entrySet()) {
            System.out.println(entry);
        }
        data.clear();
    }

    @Test(singleThreaded = true, invocationCount = 100, invocationTimeOut = 10000)
    public void testMapOperationsSafe() throws Exception {
        data.put("1", "111");
        data.put("2", "111");
        data.put("3", "111");
        data.put("4", "111");
        data.put("5", "111");
        data.put("6", "111");
        data.put("7", "111");
        for (Map.Entry<String, String> entry : data.entrySet()) {
            System.out.println(entry);
        }
        data.clear();
    }
}
```

Многопоточность



- `threadPoolSize` определяет максимальное количество потоков используемое для тестов.
- `singleThreaded` если установлен в `true` все тесты будут запущены в одном потоке.
- `invocationCount` определяет количество запусков теста.
- `invocationTimeout` определяет общее время всех запусков теста, после которого тест считается провалившимся.

Первый тест будет время от времени проваливаться с `ConcurrentModificationException`, так как будет запускаться из разных потоков, второй — нет, так как все тесты будут запущены последовательно из одного потока.

Еще можно установить параметр `parallel` у дата провайдера в `true`, тогда тесты для каждого набора данных будут запущены параллельно, в отдельном потоке:

МНОГОПОТОЧНОСТЬ



```
public class ConcurrencyTest extends Assert {
    // some staff here
    @DataProvider(parallel = true)
    public Object[][] concurrencyData() {
        return new Object[][] {
            {"1", "2"},
            {"3", "4"},
            {"5", "6"},
            {"7", "8"},
            {"9", "10"},
            {"11", "12"},
            {"13", "14"},
            {"15", "16"},
            {"17", "18"},
            {"19", "20"},
        };
    }

    @Test(dataProvider = "concurrencyData")
    public void testParallelData(String first, String
second) {
        final Thread thread = Thread.currentThread();
        System.out.printf("#%d %s: %s : %s",
thread.getId(), thread.getName(), first, second);
        System.out.println();
    }
}
```

Данный тест будет выводить нечто вроде:

```
#16 pool-1-thread-3: 5 : 6
#19 pool-1-thread-6: 11 : 12
#14 pool-1-thread-1: 1 : 2
#22 pool-1-thread-9: 17 : 18
#20 pool-1-thread-7: 13 : 14
#18 pool-1-thread-5: 9 : 10
#15 pool-1-thread-2: 3 : 4
#17 pool-1-thread-4: 7 : 8
#21 pool-1-thread-8: 15 : 16
#23 pool-1-thread-10: 19 : 20
```

Без этого параметра будет что-то вроде:

```
#1 main: 1 : 2
#1 main: 3 : 4
#1 main: 5 : 6
#1 main: 7 : 8
#1 main: 9 : 10
#1 main: 11 : 12
#1 main: 13 : 14
#1 main: 15 : 16
#1 main: 17 : 18
#1 main: 19 : 20
```


Дополнительные возможности



Кроме всего описанного есть и другие возможности, например для проверки выброса исключений(очень удобно использовать для тестов на неправильных данных):

```
public class ExceptionTest {  
    @DataProvider  
    public Object[][] wrongData() {  
        return new Object[][] {  
            {"Hello, World!!!"},  
            {"0x245"},  
            {"1798237199878129387197238"},  
        };  
    }  
    @Test(dataProvider = "wrongData", expectedExceptions =  
        NumberFormatException.class,  
        expectedExceptionsMessageRegExp = "^For input string:  
\"(.*)\"$" )  
    public void testParse(String data) {  
        Integer.parseInt(data);  
    }  
}
```

- **expectedExceptions** задает варианты ожидаемых исключений, если они не выбрасываются, тест считается провалившимся.
- **expectedExceptionsMessageRegExp** то же что и предыдущий параметр, но задает regexp для сообщения об ошибке.

Дополнительные возможности



Еще один пример:

```
public class PrioritiesTest extends Assert {  
    private boolean firstTestExecuted;  
    @BeforeClass  
    public void setUp() throws Exception {  
        firstTestExecuted = false;  
    }  
    @Test(priority = 0)  
    public void first() {  
        assertFalse(firstTestExecuted);  
        firstTestExecuted = true;  
    }  
    @Test(priority = 1)  
    public void second() {  
        assertTrue(firstTestExecuted);  
    }  
}
```

- **priority** определяет приоритет теста внутри класса, чем меньше, тем раньше будет выполнен.

Данный пример пройдет успешно, так как сначала выполнится метод `first`, затем `second`. Если поменять приоритет у `first` на 2, тест провалится.

Дополнительные возможности



Похожее поведение будет наблюдаться также если указать зависимости у теста, например, добавим в наш тест:

```
public class PrioritiesTest extends Assert {  
    // some staff  
    @Test(dependsOnMethods = {"first"})  
    public void third() {  
        assertTrue(firstTestExecuted);  
    }  
    // some staff  
}
```

Обычно это удобно, когда один тест зависит от другого, например, Утилита1 использует Утилиту0, если Утилита0 работает неправильно, то нет смысла тестировать Утилиту1. С другой стороны зависимости также удобно использовать в @Before, @After методах, особенно для связи базового теста с наследующимся, причем иногда бывает необходимо сделать так, чтобы даже если метод А свалился, а метод В зависит от него, метод В все равно вызывался. В этом случае устанавливаем параметр alwaysRun в true.

Расширение функционала



Расширение функционала может быть реализовано с помощью механизма слушателей. Поддерживаются следующие типы слушателей:

IAnnotationTransformer, IAnnotationTransformer2 — позволяют переопределять настройки теста, например, количество потоков для запуска теста, таймаут, ожидаемое исключение:

```
public class ExpectTransformer implements
IAnnotationTransformer {
    public void transform(ITestAnnotation
annotation, Class testClass, Constructor
testConstructor, Method testMethod) {
        if (testMethod.getName().startsWith("expect"))
        {
            annotation.setExpectedExceptions(new Class[]
{Exception.class});
        }
    }
}
```

Расширение функционала



Данный пример будет ожидать выброс исключения от тест-методов, начинающихся с expect.

- IHookable — позволяет переопределить тест-метод или по возможности пропустить, в tutorialе к TestNG приводится пример с JAAS.
- IMethodInterceptor, IMethodInterceptor2 — похож на предыдущий слушатель, но исполняет код до и после исполнения тест-метода
- IReporter — позволяет расширить функционал, выполняемый после выполнения всех тестов, обычно этот функционал связан с генерацией отчетов об ошибках и т.д. Таким образом можно реализовать свой механизм отчетов
- ITestListener — слушатель, который может обрабатывать большинство событий от тест-метода, например, start, finish, success, failure
- ISuiteListener — похож на предыдущий, но для сьюит, получает только события start и finish

Конфигурация



Теперь перейдем к конфигурации тестов. Простейший способ запустить тесты выглядит примерно так:

```
final TestNG testNG = new TestNG(true);
testNG.setTestClasses(new Class[] {SuperMegaTest.class});
testNG.setExcludedGroups("optional");
testNG.run();
```

Но чаще всего для запуска тестов используется XML либо YAML конфигурация. XML конфигурация выглядит примерно так:

```
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="Test suite" parallel="classes" thread-count="10">
  <test name="Default tests" verbose="1" annotations="JDK" thread-count="10"
parallel="classes">
    <parameter name="driver" value="com.mysql.jdbc.Driver"/>
    <parameter name="url" value="jdbc:mysql://localhost:3306/db"/>
    <groups>
      <run>
        <exclude name="integration"/>
      </run>
    </groups>
    <packages>
      <package name="com.example.*"/>
    </packages>
  </test>
  <test name="Integration tests" annotations="JDK">
    <groups>
      <run>
        <include name="integration"/>
      </run>
    </groups>
    <packages>
      <package name="com.example.*"/>
    </packages>
  </test>
</suite>
```


Конфигурация



Аналогичная YAML конфигурация:

```
name: YAML Test suite
parallel: classes
threadCount: 10
tests:
  - name: Default tests
    parameters:
      driver: com.mysql.jdbc.Driver
      url: jdbc:mysql://localhost:3306/db
      excludedGroups: [ integration ]
      packages:
        - com.example.*
  - name: Integration tests
    parameters:
      driver: com.mysql.jdbc.Driver
      url: jdbc:mysql://localhost:3306/db
      includedGroups: [ integration ]
      packages:
        - com.example.*
```

Тогда для запуска тестов нужно будет сделать следующее:

```
final TestNG testNG = new TestNG(true);
//final Parser parser = new
Parser("testing/testing-testng/src/test/resources/testng.xml");
final Parser parser = new
Parser("testing/testing-testng/src/test/resources/testng.yaml");
final List<XmlSuite> suites = parser.parseToList();
testNG.setXmlSuites(suites);
testNG.run();
```

Впрочем, программный запуск тестов — это излишне, т.к. для этого можно использовать возможности IDE.

Конфигурация



Вернемся к самой конфигурации. На самом верхнем уровне настраивается последовательность тестов(сюита). Может принимать следующие параметры:

- name — название используемое в отчете
- thread-count — количество потоков используемое для запуска тестов
- data-provider-thread-count — количество потоков используемое для передачи данных из дата-провайдеров в сами тесты для параллельных дата провайдеров(@DataProvider(parallel = true))
- parallel — может принимать следующие значения:
- methods — тестовые методы будут запущены в разных потоках, нужно быть осторожным если есть зависимости между методами
- classes — все методы одного класса в одном потоке, но разные классы в разных потоках
- tests — все методы одного теста в одном потоке, разные тесты в разных потоках
- time-out — время, после которого тест будет считаться провалившимся, то же что и в аннотации, но распространяется на все тестовые методы
- junit — JUnit 3 тесты
- annotations — если javadoc, то будет использован доклет для конфигурации

Конфигурация



Также могут быть настроены:

- `parameter` — параметры, те, что используются в `@Parameters`
- `packages` — пакеты, где искать тест-классы
- `listeners` — слушатели, с их помощью можно расширить функционал TestNG
- `method-selectors` — селекторы для тестов, должны реализовывать интерфейс `IMethodSelector`
- `suite-files` — можно включать другие файлы конфигурации

Сюиты в свою очередь могут включать в себя тесты с практически такими же настройками, что и для сьюит (атрибуты `name`, `thread-count`, `parallel`, `time-out`, `junit`, `annotations`, тэги `parameter`, `packages`, `method-selectors`).

Конфигурация



Также у тестов имеются и своеобразные настройки, например, запускаемые группы:

```
<test name="Default tests" verbose="1" annotations="JDK"
thread-count="10" parallel="classes">
  <!-- some staff here -->
  <groups>
    <run>
      <exclude name="integration"/>
    </run>
  </groups>
  <!-- some staff here -->
</test>
```

В данном примере тест будет включать в себя только тесты не относящиеся к группе integration.

Конфигурация



Еще тесты могут включать в себя тест-классы, которые в свою очередь могут включать/исключать в себя тест-методы.

```
<test name="Integration tests">
  <groups>
    <run>
      <include name="integration"/>
    </run>
  </groups>
  <classes>
    <class name="com.example.PrioritiesTest">
      <methods>
        <exclude name="third"/>
      </methods>
    </class>
  </classes>
</test>
```


Практическое задание



Создайте 2 ветки: **Lesson_12_junit_5** и **Lesson_12_testng**.

Напишите программу, позволяющую вычислить факториал числа. Эта программа должна быть в каждой ветке.

В ветке **Lesson_12_junit_5** напишите юнит-тесты для этой программы, используя Junit 5.

В ветке **Lesson_12_testng** напишите юнит-тесты, используя TestNG.